AI Computer Vision for Abnormal Detection in Home Pet Care Monitoring

Ng Tsz Chiu
HKU Department of Electrical and Electronic Engineering, 3036140210@connect.hku.hk

***Abstract* — This paper presents an integrated hardware-software system for real-time pet care monitoring that combines artificial intelligence (AI)-powered computer vision with Bluetooth Low Energy (BLE)-based proximity detection. A YOLOv8s object detection model, fine-tuned on a custom dataset of 4,220 annotated images, detects humans and cats simultaneously, achieving a mean average precision (mAP) at 50% intersection over union (IoU) of 92.5%, with 94.5% precision and 85.4% recall. Per-class results indicate 94.2% mAP@50 for human detection and 90.8% for cat detection. A chest-mounted wearable device integrating an ESP32- S3 microcontroller, OV5640 camera module, and dual NRF52810 BLE beacons captures video at 10 frames per second (FPS) while monitoring proximity to restricted zones. A Python Flask backend server performs real-time inference and streams annotated video to the PetGuard iOS application via the WebSocket protocol with under 20 ms latency. Multi-zone proximity alerts are delivered within 300–400 ms when the operator enters within 1 meter of a beacon-defined zone. A dual-layer thermal management system, combining passive aluminum cooling fins with active fans, extends continuous operation from approximately 10 minutes to over 2 hours. End-to-end integration testing validates the correct coordination of detection, alerting, and recording across all designed scenarios. The system demonstrates the feasibility of embedded AI monitoring for pet care accountability in residential environments.**

Index Terms — Artificial Intelligence, Bluetooth Low Energy, Computer Vision, ESP32, Object Detection, Pet Monitoring, Real-time Streaming, Wearable Device, YOLOv8.

I. INTRODUCTION

A. Background and Motivation

The professional pet care industry has experienced significant growth driven by changing attitudes toward pet ownership, with pets increasingly regarded as family members requiring specialized care [1, 2]. Professional in-home pet sitting services address limitations of pet hotels and informal arrangements by providing personalized care in familiar environments. However, most services lack monitoring mechanisms that allow pet owners to verify caregiver conduct and ensure household boundaries are respected during home visits. This accountability gap prevents many pet owners from fully trusting professional pet sitting services, particularly in emerging markets such as Hong Kong where such services are still developing. An effective monitoring solution would protect animal welfare through automated detection of human–pet interactions, enhance home security through real-time restricted zone alerts, and build trust between pet owners and service providers through transparent video records.

B. Literature Review

Existing computer vision solutions for pet monitoring focus primarily on pet behavior analysis rather than staff accountability. Hussain et al. [3] demonstrated a deep learning approach for cat activity classification using accelerometer, gyroscope, and magnetometer sensor fusion, achieving classification of activities such as walking, running, and resting. However, this work addresses pet well-being monitoring rather than caregiver oversight. The YOLOv8 family of single-stage object detectors employs Coarse-to-Fine (C2f) feature extraction modules [4]. Lin et al. [5] established the COCO benchmark dataset, providing pretrained weights that facilitate transfer learning for domain specific detection tasks. Tiwari et al. [6] proposed real-time Received Signal Strength Indication (RSSI) compensation methods for precise distance calculation in wearable applications. Despite these advances, a significant gap exists between available technology and the specific accountability needs of pet owners during professional home visits: (1) no real-time oversight of caregiver–pet interactions exists, (2) no mechanism detects unauthorized entry into restricted household zones, (3) no objective video evidence is available for dispute

resolution, and (4) no integrated wearable monitoring system has been designed for dynamic home environments.

C. Objectives and Deliverables

This paper presents a system that addresses the identified gap by integrating four components: (1) a YOLOv8s object detection model for real-time human–cat recognition, (2) a chest-mounted wearable device with an embedded camera and BLE scanning, (3) a backend server for AI inference and data management, and (4) an iOS application called PetGuard for live monitoring and multi-zone proximity alerts. The specific objectives and deliverables are:

1. Development of an AI model achieving >90% mAP@50 for real-time dual-class detection (deliverable: trained YOLOv8s model weights).

2. Construction of a wearable hardware platform supporting >2 hours of continuous operation (deliverable: chest-mounted device with thermal management).

3. Implementation of a low-latency streaming architecture with <20 ms frame delivery (deliverable: Flask backend server with WebSocket pipeline).

4. Design of a mobile application for live monitoring and zone-based alerts (deliverable: PetGuard iOS application).

5. Validation of RSSI-based multi-zone proximity detection with sub-second alert delivery (deliverable: dual BLE beacon configuration with tested alert system).

D. Paper Overview

This paper is organized as follows. Section II describes system methodology, including model training, hardware design, backend architecture, and application development. Section III presents the experimental results. Section IV provides discussion and analysis of the findings, including difficulties encountered and mitigations applied. Section V addresses limitations and proposes future work. Section VI concludes the paper by summarizing objectives achieved and the significance of the results.

II. METHODOLOGY

A. AI Model Development

1. Dataset and Preprocessing. A custom dataset of 4,220 annotated images was assembled using the Roboflow annotation platform (Figure 1) and split into training (3,736 images), validation (307 images), and testing (177 images) sets with two classes: "human" and "cat." Images were collected from diverse cat breeds and various human body configurations to promote generalization. The Roboflow platform was selected for its native support for YOLOv8 training pipelines and automated augmentation capabilities.

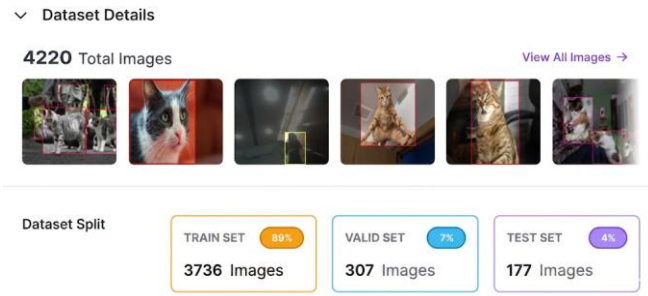

Figure 1. Dataset created on Roboflow platform

2. Model Architecture. The YOLOv8s (small) variant was selected as the detection backbone for three reasons. First, as a single-stage detector that processes images in one complete pass rather than multiple sequential stages, it processes an entire image in one forward pass, enabling real-time inference. Second, C2f modules which are feature extraction layers that identify visual patterns at multiple resolutions extract multi-scale features that improve detection of objects at varying sizes [7]. Third, with 11.1 million parameters and 28.6 GFLOPs (a computational complexity measure representing billion floating-point operations per second), the small variant balances accuracy and computational efficiency for resource-constrained deployment. Transfer learning, a technique that reuses pre-trained knowledge from existing models, was applied via pre-trained COCO dataset weights [5] to accelerate convergence. Training Configuration. The model was trained on an NVIDIA RTX 4090 GPU with 24 GB VRAM and 16,384 CUDA cores [8]. Training proceeded for 100 epochs with a batch size of 16 and an input resolution of 640 × 640 pixels. An initial batch size of 32 caused out-of-memory errors; reducing to 16 maintained stable gradient estimates within GPU memory constraints. The AdamW, an algorithm that adjusts model parameters to minimize errors during training, optimizer was employed with an initial learning rate of 0.01 (auto-adjusted to 0.001667), a final

learning rate multiplier of 0.01, and a 3-epoch warm-up period followed by linear decay (gradually reducing the rate of parameter adjustments)[9]. Early stopping was configured with a patience of 20 epochs but was not triggered, as the model continued improving through all 100 epochs. Continued training beyond an initial plateau at epoch 29 yielded an additional 2–3% mAP improvement, justifying the full 100-epoch schedule.

3. Data Augmentation. A multi-modal augmentation strategy was employed to improve model generalization (the model's ability to perform well on new, unseen data). Geometric transformations included horizontal flipping (50% probability), vertical flipping (10%), rotation (±15°), translation (±10% image size), scaling (0.8–1.2×), and shear (±10°). Color space modifications included hue adjustment (±5%), saturation (±20%) and brightness value (±20%). A 4-image mosaic composition (which combines multiple images into one training sample) at 0.5 probability increased per-batch diversity, and random erasing (a technique that removes portions of training images) at 10% probability improved robustness to partial occlusion [9, 10]. These transformations simulate the varied pet postures, lighting conditions, and camera angles encountered in residential monitoring environments.

B. Hardware System Design

1. Wearable Form Factor. The device was configured as a chest-mounted wearable (Figure 2) with a total weight of approximately 500 g. The platform (Figure 3) integrates an ESP32-S3 microcontroller featuring an Xtensa dual-core LX7 processor running at 240 MHz, 512 KB of static random-access Memory (SRAM, which provides fast temporary storage for active computations), and integrated 2.4 GHz Wi-Fi and Bluetooth 5 (Low Energy) connectivity [11]. An OV5640 5-megapixel camera module [12] was connected via the Digital Video Port (DVP) interface, with an adjustable mount providing a 0° to 180° viewing range. The wearable form factor was chosen to simulate realistic deployment conditions during professional pet-sitting sessions.

Figure 2. Chest-mount harness

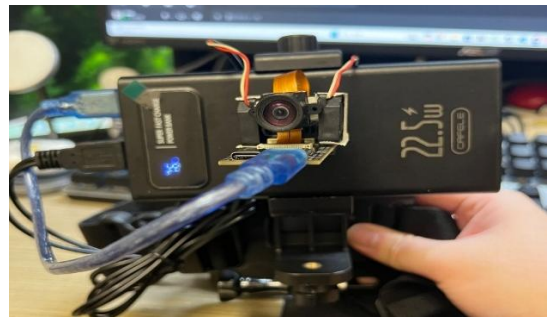

Figure 3. Chest-mount harness with integrated ESP32-S3, OV5640 camera module, and cooling components secured for wearable deployment.

2. **Thermal Management.** A dual-layer thermal management system was implemented to address thermal challenges during extended operation (Figure 4). The passive layer consists of aluminum cooling fins bonded to the ESP32-S3 and OV5640 surfaces using thermally conductive adhesive pads, increasing surface area for heat dissipation at zero power cost. The active layer comprises two 40 × 40 × 10 mm fans operating at 5 V and drawing approximately 50 mA each. This design was motivated by the observation that the device experienced thermal shutdown within 8–12 minutes during initial testing without thermal management—a critical limitation that necessitated the dual-layer redesign.

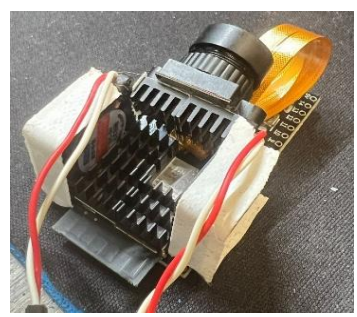

Figure 4. ESP32-S3 added cooling fins and fans

3. Power Management. A unified 10,000 mAh power bank supplies the entire system at approximately 305 mA total draw: the ESP32-S3 consumes approximately 120 mA with Wi-Fi active, the OV5640 camera draws approximately 80 mA, the dual fans draw a combined 100 mA, and miscellaneous components consume approximately 5 mA. Field testing indicated approximately 16% battery depletion over 2 hours of continuous operation, yielding an estimated practical runtime of approximately 12.5 hours.

4. BLE Beacon Configuration. Two NRF52810-based BLE beacons were configured with a broadcast interval of 960 ms at −8 dBm TX power (Mode 4) [13]. Each beacon was assigned to a specific room for multi-zone coverage. Distance estimation employs a log-distance path loss model:

$$\text{Distance} = 10^{\frac{\text{TX Power}-\text{RSSI}}{10\times n}} \quad (1)$$

where distance is the estimated distance in meters, TX power is the calibrated TX power at 1 m (−8 dBm), RSSI is the received signal strength measured by the ESP32-S3, and n = 2.5 is the path loss exponent (a coefficient that describes how signal strength decays over distance) for indoor residential environments [6]. A proximity threshold of distance ≤ 1.0 m triggers zone alerts. Beacon identification employs a dual-matching strategy: primary matching by MAC address with a UUID, Major, and Minor fallback for RF (Radio Frequency) interference environments.

5. Firmware Design. The ESP32-S3 firmware handles simultaneous video capture, BLE beacon scanning, and Wi-Fi data transmission. RTOS (Real-Time Operating System) task prioritization was implemented to prevent timing conflicts between these concurrent operations. On-device JPEG encoding at 10 FPS limited the practical frame rate from the OV5640's native 30 FPS capability to 10 FPS, due to the 512 KB SRAM constraint.

C. Backend Server Architecture

The backend server implements a hybrid HTTP and WebSocket architecture using Python Flask with the Ultralytics YOLOv8 library [4]. The overall architecture is illustrated in Figure 5.

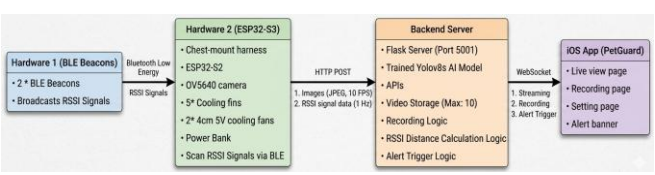

Figure 5. Backend system architecture

1. Inference Pipeline. JPEG frames received from the ESP32- S3 via HTTP POST to the /esp32/frame endpoint at 10 FPS are decoded into NumPy arrays using OpenCV, processed through the YOLOv8s model, and annotated with bounding boxes and class labels. Annotated frames are re-encoded to JPEG at quality level 75 and stored in a circular buffer, a fixed-size memory structure that overwrites the oldest data when full. Frames are broadcast to connected clients via /ws/stream as binary data consisting of a 4-byte littleendian size header followed by the raw JPEG payload, eliminating the 33% size inflation of base64 encoding (a text-based representation format).

2. Proximity State Management. Beacon distance data received via HTTP POST to /esp32/distance is evaluated against the 1.0-meter threshold. A room mapping dictionary converts beacon identifiers to human-readable room labels (e.g., "Room 1," "Room 2"). When any beacon distance falls below the threshold, the proximity_alert flag is activated and the corresponding room label(s) are included in the WebSocket payload transmitted to the PetGuard application.

3. Adaptive Frame Rate Control. A queue depth monitoring mechanism dynamically adjusts the transmission rate between 10 and 25 FPS. When queue depth exceeds 75% capacity, the frame rate is reduced to prevent backlog; when below 25%, the rate increases to maximize visual quality. This mechanism was implemented after initial testing revealed that frame accumulation during network congestion caused stuttering and delays exceeding 2 seconds.

4. Recording Logic. Video recording is triggered automatically upon simultaneous detection of a human and a cat, or upon a proximity breach event. Recordings are saved at 30 FPS with a 2-second cooldown period after the triggering condition ceases.

D. PetGuard iOS Application

The PetGuard iOS application was developed using the SwiftUI framework (Figures 6–7). A central NetworkManager class manages all backend communication via REST APIs and WebSocket connections. The Live Stream tab polls the /stream/live endpoint at 100 ms intervals (approximately 10 FPS), receiving base64-encoded JPEG frames decoded into UIImage objects for display. A top overlay bar presents the connection status, a proximity alert banner with room-specific labels, and a recording indicator. The Recordings tab supports variable-speed playback at 0.5×, 1×, and 2× rates. A Settings modal allows the user to configure the backend server IP address and port. Image decoding was offloaded to a background thread using Grand Central Dispatch (GCD) to prevent interface freezes.

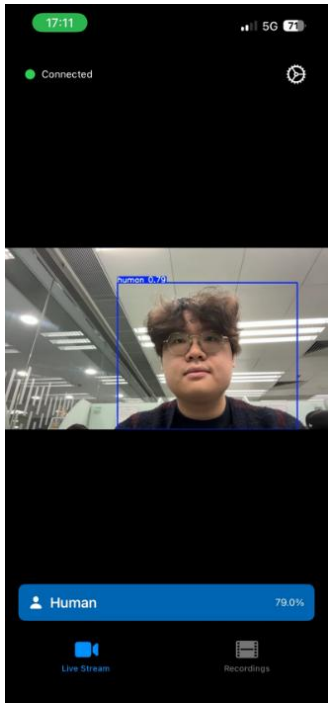

Figure 6. Live streams page of iOS app

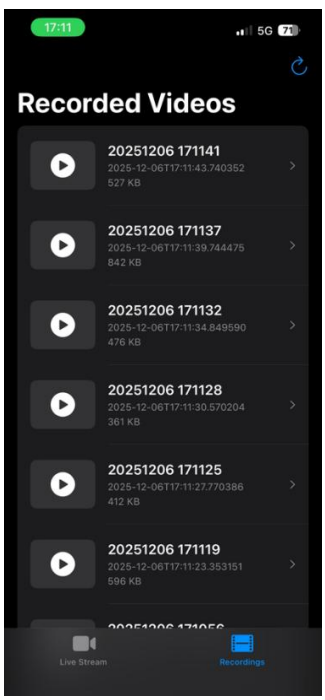

Figure 7. Recordings page of iOS app

III. EXPERIMENTAL RESULTS

A. Model Training Performance

1. **Training Results.** The model was trained for 100 epochs (total time: 1.659 hours, 60 seconds per epoch, 5.6 iterations/second) on the RTX 4090. Terminal training logs confirmed per-epoch convergence tracking (Figure 8) and final validation (Figure 9).

Figure 8. Tracking panel in terminal while training

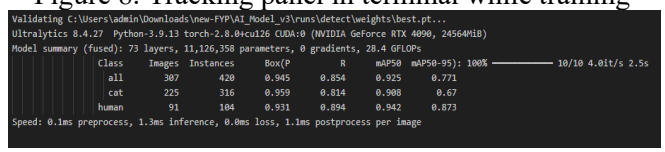

Figure 9. Terminal output once finished the training

The final model (weights/best.pt) achieved mAP@50 of 0.925 (92.5%), mAP@50-95 of 0.771 (77.1%), precision of 0.945 (94.5%), and recall of 0.854 (85.4%), validated on 420 instances across 307 images. The 92.5% mAP@50 exceeds the minimum threshold for reliable real-time pet monitoring. The high precision (94.5%) is particularly important for a system triggering recording and alerts: false alarms erode user trust.

Difficulties: Initial batch size 32 caused out-of-memory errors on the RTX 4090; reducing to 16 increased training time from an estimated 1.2 hours to 1.659 hours. Early stopping at epoch 29 was considered, but continuing to epoch 100 yielded an additional 2–3% mAP improvement.

2. Training Progression Analysis. The training history reveals a four-phase learning curve consistent with transfer-learning fine-tuning on domainspecific datasets.

Early Epochs (1–5): mAP@50 improved rapidly from 0.456 to 0.693 (Figure 10), reflecting rapid adaptation of COCO pre-trained features to the human–cat domain.

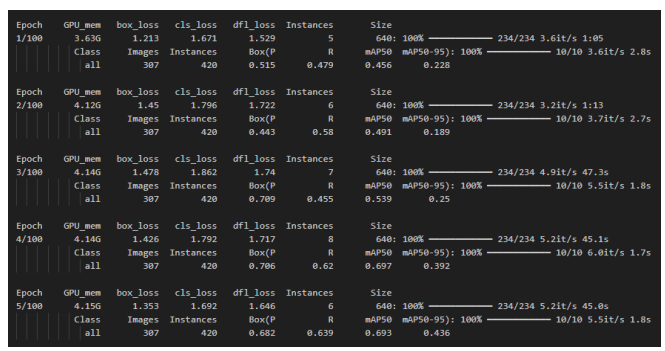

Figure 10. Result of Early Epochs (1–5)

Mid Epochs (6–15): mAP@50 climbed from 0.799 to 0.834 (Figure 11), with precision reaching 0.809 and

recall 0.763 as the model learned discriminative human–cat features beyond generic COCO representations.

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 6/100      4.19G      1.305      1.615      1.616          6        640: 100%  ━━━━━━━━━━━━ 234/234 5.1it/s 45.4s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.0it/s 2.0s
             all        307        420      0.777      0.752      0.799      0.521

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 7/100      4.22G      1.285      1.529      1.595          4        640: 100%  ━━━━━━━━━━━━ 234/234 5.1it/s 46.1s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
             all        307        420      0.791      0.681      0.774      0.495

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 8/100      4.27G      1.246      1.506      1.569          3        640: 100%  ━━━━━━━━━━━━ 234/234 5.0it/s 46.9s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.3it/s 1.9s
             all        307        420      0.785      0.756      0.794      0.526

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 9/100      4.29G      1.211      1.451      1.542          6        640: 100%  ━━━━━━━━━━━━ 234/234 5.0it/s 46.9s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 6.1it/s 1.6s
             all        307        420      0.854      0.724      0.812      0.567

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
10/100      4.34G      1.195      1.412      1.533          7        640: 100%  ━━━━━━━━━━━━ 234/234 5.1it/s 46.2s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.9it/s 1.7s
             all        307        420      0.838      0.741      0.816      0.567

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
11/100      4.36G      1.162      1.344      1.504          2        640: 100%  ━━━━━━━━━━━━ 234/234 5.2it/s 45.4s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.1it/s 2.0s
             all        307        420      0.819      0.734      0.828      0.582

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
12/100      4.41G      1.157      1.332      1.506          4        640: 100%  ━━━━━━━━━━━━ 234/234 5.1it/s 46.3s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.9it/s 1.7s
             all        307        420      0.838      0.758      0.836      0.593

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
13/100      4.41G      1.126        1.3      1.475          4        640: 100%  ━━━━━━━━━━━━ 234/234 5.2it/s 45.0s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 6.1it/s 1.6s
             all        307        420      0.812      0.687       0.83      0.615

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
14/100      4.41G      1.126      1.264      1.472          4        640: 100%  ━━━━━━━━━━━━ 234/234 4.7it/s 49.7s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
             all        307        420      0.867      0.749      0.855      0.622

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
15/100      4.41G      1.111      1.249      1.461          6        640: 100%  ━━━━━━━━━━━━ 234/234 5.0it/s 47.0s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 6.1it/s 1.6s
             all        307        420      0.809      0.763      0.834      0.583
```

Figure 11. Result of Mid Epochs (6–15)

Later Epochs (16–39): mAP@50 progressed from 0.837 to 0.897 (Figures 12-13), with precision rising to 0.908 while recall stabilized above 0.80, indicating the model adopted a more conservative, higher-confidence detection strategy.

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
16/100      4.41G      1.113      1.234      1.462          9        640: 100%  ━━━━━━━━━━━━ 234/234 4.1it/s 56.6s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 4.2it/s 2.4s
             all        307        420      0.813      0.776      0.837      0.612

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
17/100      4.41G      1.078      1.208      1.448          5        640: 100%  ━━━━━━━━━━━━ 234/234 3.3it/s 1:10
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.9it/s 2.5s
             all        307        420      0.902      0.774      0.858      0.637

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
18/100      4.41G      1.063      1.156      1.429          3        640: 100%  ━━━━━━━━━━━━ 234/234 3.3it/s 1:12
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.5it/s 2.9s
             all        307        420      0.864      0.787      0.853      0.648

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
19/100      4.41G      1.061      1.143      1.427          9        640: 100%  ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.916      0.786      0.878      0.658

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
20/100      4.41G      1.055      1.132      1.417          8        640: 100%  ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.6it/s 2.7s
```

Figure 12. Result of Later Epochs (16–20)

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
38/100      4.41G      0.923     0.9277      1.319          7        640: 100%  ━━━━━━━━━━━━ 234/234 3.6it/s 1:05
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.8it/s 2.6s
             all        307        420      0.889      0.801      0.887        0.7

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
39/100      4.41G     0.9119     0.9058      1.314          6        640: 100%  ━━━━━━━━━━━━ 234/234 3.2it/s 1:14
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.9it/s 2.6s
             all        307        420      0.908      0.802      0.897      0.714
```

Figure 13. Result of Later Epochs (38–39)

Final Epochs (40–100): Peak mAP@50 of 0.924 was reached at epoch 100 (Figures 14–15). A temporary plateau between epochs 40–50 initially raised convergence concerns, but continued training with linear LR decay enabled the optimizer to escape shallow local minima.

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
40/100      4.41G     0.9146      0.914      1.317         10        640: 100%  ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.899      0.808      0.892      0.697

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
41/100      4.41G     0.8964     0.8828      1.303         10        640: 100%  ━━━━━━━━━━━━ 234/234 3.3it/s 1:11
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.8it/s 2.7s
             all        307        420       0.91      0.835      0.901      0.713

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
42/100      4.41G     0.9042     0.8888      1.305          9        640: 100%  ━━━━━━━━━━━━ 234/234 3.3it/s 1:12
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.6it/s 2.8s
             all        307        420      0.893      0.839      0.905      0.717

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
43/100      4.41G     0.9051     0.8846      1.306          9        640: 100%  ━━━━━━━━━━━━ 234/234 3.3it/s 1:12
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.893      0.861      0.903       0.72
```

Figure 14. Result of Final Epochs (40–43)

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
97/100      4.41G     0.5219     0.3752      1.038          2        640: 100%  ━━━━━━━━━━━━ 234/234 5.7it/s 40.9s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.7it/s 1.8s
             all        307        420      0.942      0.857      0.921      0.767

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
98/100      4.41G     0.5294     0.3822      1.045          3        640: 100%  ━━━━━━━━━━━━ 234/234 5.6it/s 42.1s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 6.0it/s 1.7s
             all        307        420      0.945       0.86      0.924      0.768

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
99/100      4.41G     0.5301     0.3914      1.045          2        640: 100%  ━━━━━━━━━━━━ 234/234 5.7it/s 41.0s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.7it/s 1.7s
             all        307        420       0.94      0.863      0.927      0.769

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
100/100     4.41G     0.5269     0.3715       1.04          9        640: 100%  ━━━━━━━━━━━━ 234/234 5.6it/s 41.5s
           Class     Images  Instances     Box(P          R     mAP50  mAP50-95): 100%  ━━━━━━━━━━━━ 10/10 5.1it/s 2.0s
             all        307        420      0.944      0.854      0.924      0.772
```

Figure 15. Result of Final Epochs (97-100)

B. Per-Class Detection Accuracy

The final per-class detection performance, validated on 420 instances across 307 validation images, is presented in Table 1. The overall model achieved 92.5% mAP@50, exceeding the 90% target established in the project objectives.

Table 1: Per-Class Detection Accuracy

Class	**Prec.**	**Recall**	**mAP @50**	**mAP @50-95**	**Inst.**
Cat	0.959	0.814	0.908	0.670	316
Human	0.931	0.894	0.942	0.873	104
Overall	**0.945**	**0.854**	**0.925**	**0.771**	**420**

C. Real-World Hardware Validation

Qualitative validation was performed on the ESP32-S3 and OV5640 hardware platform. As shown in Figure 16, both human and cat detections were correctly identified with appropriate confidence scores under adequate lighting, confirming that the model generalizes from the training environment to the target embedded hardware.

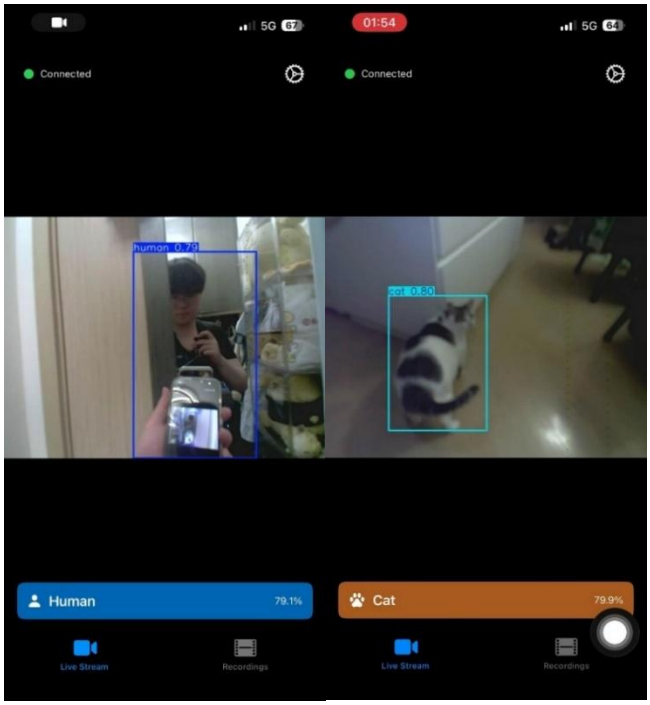

Figure 16. Real-world validation on the ESP32-S3 and OV5640 hardware platform: (left) human detection and (right) cat detection

Two environmental limitations were identified, as illustrated in Figure 17. In low-light conditions, confidence scores degraded to the range of 0.4–0.6 with occasional complete detection failures. Additionally, cats positioned beyond 1 meter from the camera were frequently missed due to the resolution constraints of the 640 × 640 pixel input.

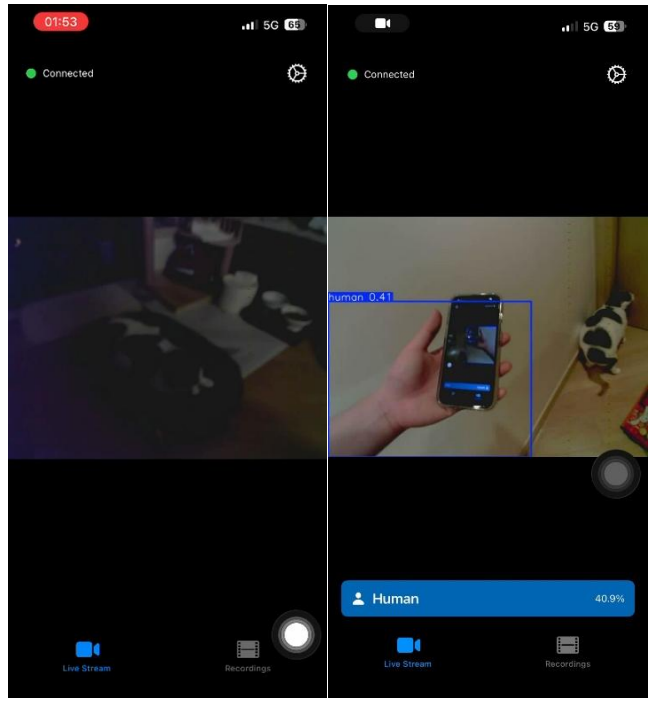

Figure 17. Model failure cases: in (left) low-light conditions & (right) missed detection

D. Hardware Performance

Table 2 summarizes the measured hardware performance metrics. The dual-layer thermal management system achieved a 12-fold improvement in continuous operation time, extending runtime from approximately 10 minutes to over 2 hours at ambient temperatures of 22–25°C.

Table 2. Hardware System Performance

Metric	Value
Device Weight	≈500 g
Total Power Draw	305 mA at 5 V
Battery Capacity	10,000 mAh
Continuous Runtime	>2 hours (verified)
Without Cooling	8–12 min (shutdown)
With Dual-Layer Cooling	>2 hours, no throttle
RSSI Precision	±1.0–1.5 m
False Alert Rate	5–8%

E. BLE Proximity Testing

The dual-beacon configuration was validated through six test scenarios, with results presented in Table 3. All scenarios passed, demonstrating correct differentiation between single-zone and dual-zone proximity events, accurate beacon-to-room-label mapping, and proper integration of proximity alerts with the recording trigger logic.

Table 3. Beacon Placement Test Results

Scenario	**B1 RSSI**	**B2 RSSI**	**Response**
At B1 (<1m)	-45 dBm	-75 dBm	"Rm 1 alert"
At B2 (<1m)	-75 dBm	-40 dBm	"Rm 2 alert"
At B1 (≈2 m)	-65 dBm	-65 dBm	No alert
Exit B1	-75 dBm	-76 dBm	Stop alert
At B1 & B2	-48dBm	-42 dBm	"Rm 1&2 alert"

F. System Latency

Table 4 presents the measured end-to-end latency for each pipeline stage. A complete end-to-end integration test (Figure 18) verified the full detection–alert–recording sequence: simultaneous human and cat detection triggered recording, a BLE proximity breach triggered a room-specific alert on the PetGuard application within 300–400 ms, and recorded video became available within approximately 500 ms of recording cessation.

Table 4. System Latency Measurements

Measurement	Latency
WebSocket frame delivery	<20 ms
RSSI threshold to alert	300–400 ms
Detection to recording start	≈100 ms
Recording stop to availability	≈500 ms
/esp32/frame response	<50ms
Lightweight endpoints	<20ms

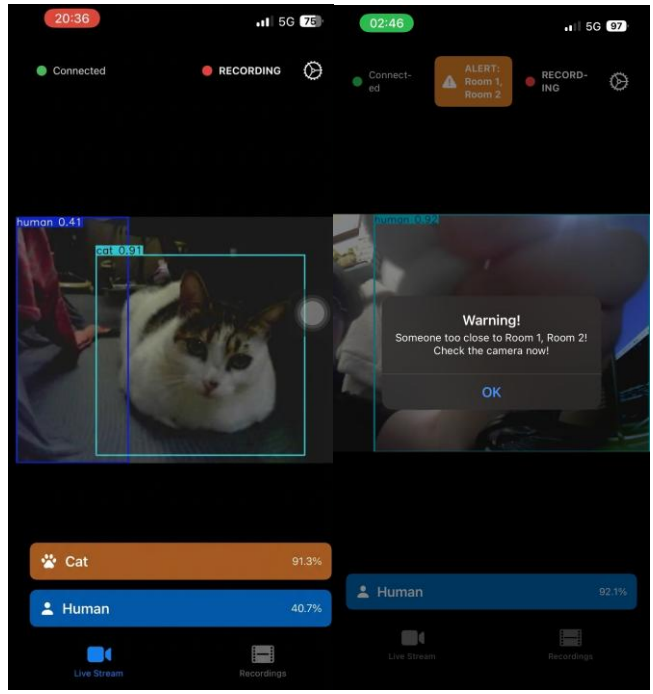

Figure 18. End-to-end integration test: simultaneous human and cat detection with recording active (left), and the proximity alert pop-up on the PetGuard iOS application when the ESP32-S3 enters within 1 meter of the BLE beacon (right)

IV. DISCUSSION

A. Detection Performance Analysis

The per-class results in Table 1 reveal a notable asymmetry between human and cat detection. The 8% recall gap (human at 89.4% versus cat at 81.4%) is attributable to the greater morphological diversity within the cat class: cats exhibit wider variation in size, fur patterns, body postures, and color compared to humans, who share a more consistent body structure. Conversely, cat precision (95.9%) exceeds human precision (93.1%), indicating fewer false positive cat detections. This precision–recall profile is favorable for the intended use case, as the cost of a false alarm—an unnecessary recording triggered by an incorrect cat detection— exceeds the cost of briefly missing a cat that is likely to be detected in subsequent frames.

The mAP@50-95 disparity (cat at 0.670 versus human at 0.873) in Table 1 indicates that cat bounding box localization is less precise across stricter IoU thresholds. This is consistent with the irregular silhouettes of cats (tails, ears, curled postures) compared to the relatively rectangular human form, making tight bounding box prediction more challenging for the cat class.

B. Environmental Limitations

As demonstrated in Figure 17, reliable operation requires adequate ambient lighting and subjects within approximately 1 meter of the camera. The low-light degradation to confidence scores of 0.4–0.6 results from two compounding factors: the training dataset contained insufficient low-light samples, and the OV5640 sensor has inherently limited lowlight sensitivity without supplemental illumination. The chest-mounted wearable form factor partially mitigates the distance limitation, as the operator is typically within arm's reach of the pet during care activities.

C. Streaming Architecture Performance

The migration from HTTP with base64-encoded JPEG to WebSocket with binary frame transmission reduced endto-end latency from 1–2 seconds to less than 20 ms, as shown in Table 4. This improvement resulted from eliminating per-frame connection overhead and reducing per-frame data size by approximately 33%. The 300–400 ms alert delivery latency is imperceptible relative to the timescale of human movement through a room.

D. Proximity Detection Accuracy

As indicated in Table 2, RSSI-based distance estimation precision degrades from ±0.5 m in open spaces to ±1.0– 1.5 m in furnished environments. This degradation is caused by multipath interference from signal reflections off metal objects, a well-documented limitation of RSSI-based localization [6]. The 5–8% false alert rate is acceptable for this prototype: the consequence of a false alert (an unnecessary notification) is minor compared to the consequence of a missed alert (an undetected zone breach). A 2-second debounce cooldown was implemented to reduce alert fatigue during rapid zone entry and exit events.

E. Difficulties Encountered and Mitigations

Several significant difficulties were encountered during system development. The most critical was thermal shutdown of the ESP32-S3 within 8–12 minutes of operation, resolved through the dual-layer cooling system described in Section II-B. An initial training batch size of 32 caused GPU out-of-memory errors, mitigated by reducing the batch to 16. The original HTTP streaming architecture exhibited 1–2 second visible delay; migration to WebSocket with binary transmission resolved this. Beacon MAC address instability in certain RF environments was addressed by implementing a UUID, Major, and Minor fallback matching strategy. Base64 image decoding on the iOS main thread caused interface freezes, resolved by offloading decoding to a background thread via GCD. RSSI oscillation near the 1-meter threshold caused repeated alert triggering, which a 2-second debounce cooldown mitigated.

F. Scalability Considerations

The Flask-based server architecture with the Python Global Interpreter Lock (GIL)—permitting only one thread to execute Python bytecode at a time—limits true parallelism. Testing indicated that performance degrades beyond approximately 3 concurrent clients. This constraint is acceptable for the current single-household deployment scenario but represents a scaling concern for future commercial deployment.

V. LIMITATIONS AND FUTURE WORK

The system exhibits limitations across several dimensions. The AI model's performance degrades in low-light conditions (confidence of 0.4–0.6) and for objects beyond 1 meter, constrained by the 4,220-image single-domain dataset and the OV5640 sensor's limited low-light sensitivity. Only 104 human instances across 91 validation images were available, limiting statistical confidence to ±5–8% confidence intervals. RSSI-based proximity estimation degrades to ±1.0–1.5 m in furnished environments with a 5–8% false alert rate, and RSSI provides distance but not direction of approach. The system operates exclusively on local Wi-Fi without cloud accessibility, cooling fans run continuously regardless of thermal demand, and the 512 KB SRAM constraint limits the frame rate to 10 FPS. All testing was conducted in a single home layout, and the PetGuard application lacks offline playback, in-application recording management, and user authentication.

Future work should address these limitations through the following enhancements: (1) expanding the training dataset to over 10,000 images with diverse lighting, cat breeds, and home environments, and implementing hard example mining to target a 30–40% reduction in false negatives; (2) integrating infrared illumination or upgrading to a camera module with improved low-light sensitivity; (3) applying Kalman filtering for RSSI measurement smoothing, targeting a reduction in false alert rate to 2–3%; (4) incorporating a DS18B20 digital temperature sensor for adaptive fan speed control, reducing power draw from 305 mA to approximately 250 mA; (5) migrating to cloud infrastructure (AWS or Microsoft Azure) with GPU-accelerated inference; (6) adding 8 MB of external

PSRAM to enable 30 FPS operation; (7) implementing per-class confidence thresholds and cosine annealing learning rate schedules; and (8) adding offline video playback, alert history logging, and configurable hysteresis thresholds (trigger at 0.9 m, clear at 1.1 m) to the PetGuard application.

VI. CONCLUSION

This paper presented an integrated AI-powered pet care monitoring system combining YOLOv8s object detection with BLE-based multi-zone proximity alerting in a wearable form factor. All five project objectives were achieved. First, the AI model attained 92.5% mAP@50 with 94.5% precision and 85.4% recall, exceeding the 90% accuracy target. Second, the wearable device operates continuously for over 2 hours with dual-layer thermal management, representing a 12-fold improvement over the uncooled configuration. Third, the WebSocket-based streaming architecture delivers annotated video at less than 20 ms latency. Fourth, the PetGuard iOS application provides stable live streaming, variable-speed recording playback, and room-specific proximity alerts. Fifth, the BLE proximity detection system delivers alerts within 300–400 ms across all six validated test scenarios.

The significance of these results lies in demonstrating that embedded AI-powered wearable monitoring is technically feasible for pet care accountability within residential environments, addressing an identified gap in current pet care technology. The system establishes a technical foundation for future development toward production-ready deployment incorporating cloud integration, improved environmental robustness, and expanded detection capabilities.

ACKNOWLEDGEMENT

The author expresses sincere gratitude to Dr. W.T. Fok for his supervision and guidance throughout this project, and to Dr. Andrew H.C. Wu for serving as second examiner. Thanks are also extended to the Department of Electrical and Electronic Engineering at the University of Hong Kong for providing laboratory access and resources.

REFERENCES

[1] "Growing global pet grooming market," Vet. Rec., vol. 196, no. 2, p. 65, Jan. 2025, doi: 10.1002/vetr.5144.

[2] Y. Brugger, "Private equity challenge – Pets at home commercial assessment of European pet market," M.S. thesis, ProQuest Dissertations & Theses Global, 2024.

[3] A. Hussain, S. Ali, M.-I. Joo, and H.-C. Kim, "A deep learning approach for detecting and classifying cat activity to monitor and improve cat's well-being using accelerometer, gyroscope, and magnetometer," IEEE Sensors J., vol. 24, no. 2, pp. 1996–2008, Jan. 2024, doi: 10.1109/JSEN.2023.3324665.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics Documentation, Jan. 2023. [Online]. Available: https://docs.ultralytics.com/models/yolov8/

[5] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740–755, doi: 10.1007/978-3-319-10602- 1 48.

[6] K. R. Tiwari, I. Singhal, and A. Mittal, "Real time RSSI compensation for precise distance calculation using sensor fusion for smart wearables," Adv. Sci. Technol. Eng. Syst. J., vol. 6, no. 4, pp. 327–333, Aug. 2021, doi: 10.25046/aj060436.

[7] Stanford University, "Lecture 9: Detection, Segmentation, Visualization, and Understanding," CS231n Course Lecture Slides, 2025. [Online]. Available: https://cs231n.stanford.edu/slides/2025/lecture_9.pdf

[8] NVIDIA, "NVIDIA Delivers Quantum Leap in Performance, Introduces New Era of Neural Rendering With GeForce RTX 40 Series," NVIDIA Newsroom, Sep. 20, 2022. [Online]. Available: https://nvidianews.nvidia.com/news/ nvidia-delivers-quantum-leap-in-performance

[9] Stanford University, "Lecture 3: Regularization and optimization," CS231n Course Lecture Slides, 2025. [Online]. Available: https://cs231n.stanford.edu/slides/2025/lecture_3.pdf

[10] Stanford University, "Lecture 12: Self-Supervised Learning," CS231n Course Lecture Slides, 2025.

[Online]. Available: https://cs231n.stanford.edu/slides/2025/lecture_12.pdf

[11] Espressif Systems, “ESP32-S3 Series Datasheet,” Version 2.2, 2021. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_ datasheet_en.pdf

[12] OmniVision Technologies, “OV5640 Datasheet,” Version 2.03, 2011. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/OV5640_datasheet.pdf

[13] Shenzhen Radioland Technology Co., Ltd., “52810-B Series Specification V1.0,” Nov. 2022.